

OLAP SQL

Las consultas de **OLAP** demandan funcionalidades de análisis aún no presentes en el **SQL avanzado** de los gestores relacionales transaccionales (**OLTP**) característicos del **operacional**. Además de los agregados, que cobran un papel protagonista, hay otros servicios algunos de los cuales se presentan a continuación

Common Table Expression (CTE)

- Una *CTE* es un conjunto de resultados temporal al que se puede hacer referencia dentro de una instrucción *SELECT*, *INSERT*, *UPDATE* o *DELETE*
- Se define al principio de la consulta utilizando la palabra clave **WITH**
- Normalmente se utiliza para mejorar la legibilidad y dividir consultas complejas en partes más pequeñas
- Las *CTEs* son temporales y solo existen durante la ejecución de la consulta

Ejemplo

```
WITH DepartmentSales AS (  
    SELECT e.department, SUM(s.amount) AS total_sales  
    FROM sales s  
    JOIN employees e ON s.employee_id = e.employee_id  
    WHERE MONTH(s.sale_date) = MONTH(CURDATE())  
    GROUP BY e.department  
)  
SELECT department, total_sales  
FROM DepartmentSales  
WHERE total_sales < 2000;
```

- **A favor:** Mejora la legibilidad al permitir que la parte `DepartmentSales` se calcule por separado
- **En contra:** Es temporal y no se puede reutilizar en varias consultas

SubQuery

- Una **subconsulta** es una consulta anidada dentro de otra consulta dentro de las cláusulas `SELECT`, `FROM` o `WHERE`

- Se puede utilizar para filtrar o calcular un conjunto de resultados basándose en el resultado de otra consulta
- Las subconsultas pueden ser **correlacionadas** (dependientes de la consulta externa) o **no correlacionadas** (independientes de la consulta externa)

Ejemplo

```
SELECT department, total_sales
FROM (
    SELECT e.department, SUM(s.amount) AS total_sales
    FROM sales s
    JOIN employees e ON s.employee_id = e.employee_id
    WHERE MONTH(s.sale_date) = MONTH(CURDATE())
    GROUP BY e.department
) AS SalesPerDepartment
WHERE total_sales < 2000;
```

- **A favor:** Las subconsultas se pueden anidar y utilizar directamente dentro de la estructura de la consulta

- **En contra:** Las subconsultas pueden provocar en ocasiones problemas de rendimiento si se utilizan en exceso en consultas complejas, lo que dificulta su optimización y comprensión

Tablas temporales

- Una **tabla temporal** es una tabla que existe de forma temporal y se almacena en la base de datos mientras dura la sesión del usuario o la transacción
- Se puede crear una tabla temporal para almacenar resultados intermedios y utilizarla en consultas posteriores
- Las tablas temporales son útiles para **almacenar datos que deben reutilizarse en varias consultas**

Ejemplo

```
CREATE TEMPORARY TABLE TempSales AS
SELECT e.department, SUM(s.amount) AS total_sales
FROM sales s
JOIN employees e ON s.employee_id = e.employee_id
WHERE MONTH(s.sale_date) = MONTH(CURDATE())
GROUP BY e.department;
SELECT department, total_sales
FROM TempSales
WHERE total_sales < 2000;
```

- **A favor:** La tabla temporal se puede utilizar varias veces en diferentes partes de las consultas, lo que la hace ideal para consultas complejas o si se necesita utilizar el resultado en varios pasos o consultas
- **En contra:** Ocupa espacio de almacenamiento temporalmente y es específica de la sesión, lo que significa que, una vez que la sesión finaliza, la tabla se elimina

Vistas

- Una **vista** es una **tabla virtual** creada por una consulta. No almacena datos por sí misma, sino que presenta datos de una o varias tablas en función de una consulta
- A diferencia de las *CTEs* o las subconsultas, **las vistas pueden ser persistentes (materializadas)** y pueden reutilizarse en diferentes sesiones y consultas
- Las vistas proporcionan una **capa de abstracción** y pueden simplificar consultas complejas mediante la creación de lógica *SQL* reutilizable

Ejemplo

```
CREATE VIEW DepartmentSalesView AS
SELECT e.department, SUM(s.amount) AS total_sales
FROM sales s
JOIN employees e ON s.employee_id = e.employee_id
WHERE MONTH(s.sale_date) = MONTH(CURDATE())
GROUP BY e.department;
SELECT department, total_sales
FROM DepartmentSalesView
WHERE total_sales < 2000;
```

- **A favor:** Las vistas son persistentes, lo que significa que se pueden utilizar repetidamente sin tener que reescribir la lógica cada vez
- **En contra:** Las vistas no son adecuadas para operaciones complejas a gran escala en las que los datos deben modificarse con frecuencia o consultarse en diferentes contextos, ya que no almacenan datos, sino que consultan las tablas subyacentes cada vez



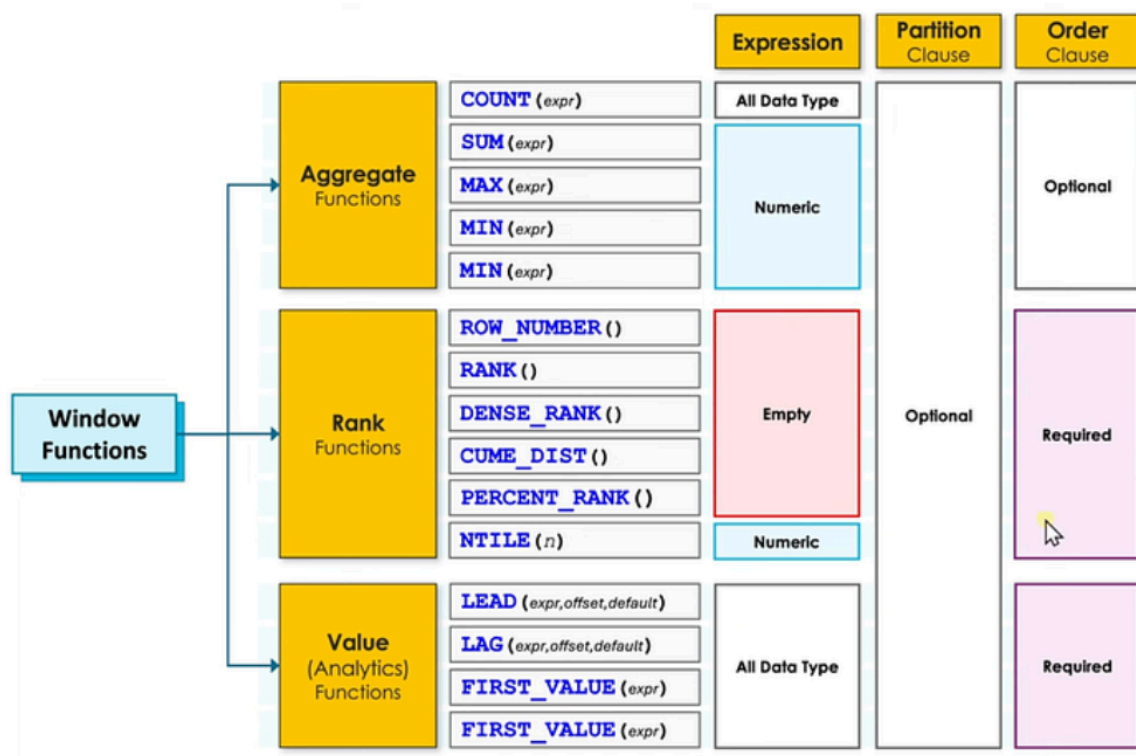
Resumen

Feature	CTE	Subquery	Temporary Table	View
Definition	Temporary result set used within a query	Query inside another query	Temporary table stored in DB	Virtual table based on a SELECT query
Scope	Limited to the query where it's defined	Limited to the outer query	Lives for the session or transaction	Persistent across sessions
Performance	Can be optimized, typically more readable	May degrade performance in complex queries	Reusable, can improve performance with complex steps	Depending on DB, views can improve or reduce performance
Reusability	Not reusable across multiple queries	Not reusable	Reusable in multiple queries during session	Reusable and simplifies complex queries
Example Usage	Breaking complex queries into simpler components	Filtering or computing intermediate results	Temporary data storage for multi-step queries	Simplifying complex queries for repeated use

WINDOW Functions

Son funciones que operan creando ventanas virtuales para realizar una operación en cada ventana. Fundamentalmente, se pueden agrupar en tres categorías:

- Funciones de agregación
- Funciones de rango o de clasificación
- Funciones analíticas o de valor



Laboratorio

Funciones de Ventanas en DuckDB

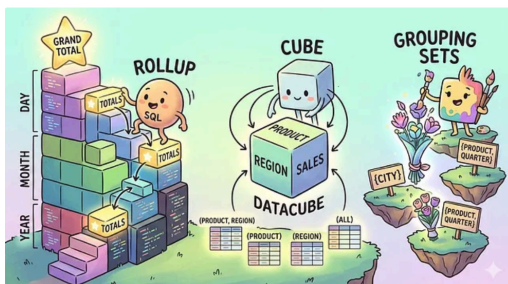
- Parte I: DDL
- Parte II: DML

GROUPING SETS, ROLLUP, CUBE

GROUPING SETS

Técnicas Avanzadas de Agregación

Ejemplo Grouping Sets



Fruit	City	Amt
🍎	NY	10
🍌	CA	15
🍎	CA	20
🍌	NY	15

Fruit	City	Sum(Amt)
-	NY	25
-	CA	35
🍎	-	30
🍌	-	15
🍌	NY	15

Select Fruit, City, SUM(Amt)
 FROM Spends
 Group By
 GROUPING SETS ((City), (Fruit));

City group
 Fruit group

GROUPING SETS permite definir varias combinaciones de agrupación en una sola consulta. Equivale a ejecutar varias consultas *GROUP BY* a la vez y apilar los resultados. De este modo es innecesario invocar *UNION*

USO INCORRECTO

```
-- Calculating Area Total
SELECT Area, NULL as Product, SUM(SalesAmount) as TotalSales
FROM Sales
UNION ALL
-- Calculating Region Total
SELECT Region, NULL, SUM(SalesAmount)
FROM Sales
GROUP BY Region
UNION ALL
-- Calculating Region + Product Total
SELECT Region, Product, SUM(SalesAmount)
FROM Sales
GROUP BY Region, Product;
```

CASE + GROUPING SETS

```
SELECT
    -- Identify the aggregation level using GROUPING()
    CASE
        WHEN GROUPING(Region) = 1 AND GROUPING(Product) = 1 THEN 'Grand Total'
        WHEN GROUPING(Area) = 1 THEN 'Region Total'
        WHEN GROUPING(Product) = 1 THEN 'Region Total'
        ELSE 'Item Level'
    END as AggregationLevel,
    COALESCE(Region, 'All Regions') as Region,
    COALESCE(Product, 'All Products') as Product,
    SUM(SalesAmount) as TotalRevenue,
    COUNT(DISTINCT OrderID) as TransactionCount
FROM Sales
GROUP BY GROUPING SETS (
    (Region, Product), -- Detailed breakdown
    (Region),          -- Region sub-totals
    (Area),            -- Area total
    ()                 -- Grand total
);
```

ROLLUP

Datos de Muestra					Ejemplo <i>ROLLUP</i>			
order_id	year	quarter	region	revenue	year	quarter	region	total_revenue
2001	2024	Q1	North	500	2024	Q1	North	500
2002	2024	Q1	South	320	2024	Q1	South	320
2003	2024	Q2	North	410	2024	Q1	NULL	820 ← Q1 subtotal
2004	2024	Q2	South	290	2024	Q2	North	410
2005	2024	Q3	North	600	2024	Q2	South	290
2006	2024	Q3	South	380	2024	Q2	NULL	700 ← Q2 subtotal
2007	2024	Q4	North	480	2024	Q3	North	600
2008	2024	Q4	South	350	2024	Q3	South	380
					2024	Q3	NULL	980 ← Q3 subtotal
					2024	Q4	North	480
					2024	Q4	South	350
					2024	Q4	NULL	830 ← Q4 subtotal
					2024	NULL	NULL	3330 ← Year total
					NULL	NULL	NULL	3330 ← Grand total

Diseñado para **datos jerárquicos**, normalmente las dimensiones lo son, calcula automáticamente los subtotales en cada nivel de la jerarquía de agrupación y añade una fila de totales al final. Consecuentemente, el orden de las columnas en *ROLLUP* es determinante ya que el cálculo siempre se realiza de derecha a izquierda

```
SELECT
  year,
  quarter,
  region,
  SUM(revenue) AS total_revenue
FROM orders
GROUP BY ROLLUP (year, quarter, region);
```



`ROLLUP(year, quarter, region)` genera automáticamente agrupaciones en cuatro niveles:

1. `(year, quarter, region)` : la fila con mayor nivel de detalle
2. `(year, quarter)` : subtotal por trimestre, agrupando las regiones
3. `(year)` : subtotal por año, agrupando los trimestres y las regiones
4. `()` : el total general

CUBE

Este operador genera todas las combinaciones de agrupación posibles, incluidos todos los subtotales y el total general. A diferencia de *ROLLUP*, que es jerárquico, *CUBE* es **exhaustivo**. Es la herramienta adecuada para producir una vista multidimensional completa de los datos